
ESTUDIO TECNICO MSX

El Chip SCC de Konami

Estudio tecnico y guia de programacion para MSX

RESUMEN

Manual exhaustivo sobre el chip de sonido SCC (Sound Creative Chip) de Konami: historia, especificaciones, mapa de registros, deteccion, tecnicas de programacion en Z80, efectos de tracker, toolchain y ecosistema. Incluye codigo comentado y referencias verificadas.

AUTOR

Estudio Mideas

FUENTES VERIFICADAS

openMSX - libmsx - BiFi/msxnet - MSX Wiki - Implementacion Mideas



Indice

Introduccion	3
Historia y contexto	3
Especificaciones tecnicas	5
Mapa de memoria y registros	7
Deteccion y activacion del chip	9
Waveforms: el corazon del SCC	11
Frecuencia y notas musicales	13
Tecnicas avanzadas: PCM y ruido	15
Motor de musica: envolventes, arpeggios y vibrato	16
Layout de datos y API publica	18
Estudio de caso: el backend SCC de Mideas	19
Toolchain y ecosistema	22
Emulacion y depuracion en openMSX	24
Apéndice A: Fuentes y referencias	25

INTRODUCCION

Introduccion

Por que este manual y como leerlo

El SCC (Sound Creative Chip) es, para muchos, la pieza de hardware que define el sonido de la era dorada de los MegaROM de Konami en MSX. Mientras el PSG (AY-3-8910) estandar entregaba tres canales de onda cuadrada y ruido, Konami anadio a sus cartuchos un sintetizador por wavetable de cinco canales que permitia timbres ricos: cuerdas, metales, bajos con cuerpo y efectos imposibles para el PSG.

Este documento es un **estudio tecnico practico**. No se queda en la descripcion: explica como activar el chip, como escribir en sus registros, como generar tablas de notas, como construir un motor de musica por filas (row player), y como depurarlo en openMSX. Toda la informacion de registros y formulas esta **cruzada con cuatro fuentes primarias**: el codigo del emulador openMSX (`src/sound/SCC.cc`), la libreria C libmsx de mori0091, las paginas tecnicas de BiFi en msxnet.org, y la implementacion nativa Z80 del proyecto Mideas.

Audiencia: programadores Z80 y desarrolladores de homebrew MSX, compositores de chiptune, y curiosos del hardware retro. No se asume conocimiento previo del SCC, pero si del ensamblador Z80 basico, del modelo de slots/paginas del MSX y del mapper Konami.

Como esta organizado

Los capitulos 1 a 3 son teoricos (historia, especificaciones y mapa de memoria). Los capitulos 4 a 9 son practicos, con codigo Z80 comentado y patrones de driver. El capitulo 10 documenta una implementacion real (el backend SCC del proyecto Mideas). Los capitulos 11 y 12 cubren el toolchain y el ecosistema. Los apendices contienen tablas de referencia lista para copiar.

CAPITULO 01

Historia y contexto

De Nemesis 2 al MegaFlashROM SCC+ SD

Que es el SCC

El acronimo SCC se ha popularizado como *Sound Custom Chip*, pero el nombre oficial, confirmado por el propio Konami en el *Konami Software Club newsletter numero 3 (noviembre 1987)*, es **Sound Creative Chip**. Ambas formas circulan en la comunidad y son aceptadas; este manual usa simplemente **SCC**.

El SCC es un chip de sonido por **wavetable**: cada canal no genera una forma de onda matematica (como la cuadrada del PSG) sino que *reproduce* una tabla de 32 muestras de 8 bits con signo almacenada en RAM, recorriendola de forma ciclica a una velocidad fijada por un divisor de frecuencia. Esto permite cualquier timbre imaginable: seno, diente de sierra, triangulo, onda compleja muestreada o ruido.

Numeros de pieza

Konami uso tres piezas de silicio distintas a lo largo de la vida comercial del SCC. Es importante distinguirlas porque sus mapas de registros y capacidades difieren:

Pieza	Nombre	Funcion
K051649	SCC original	Generador de sonido. El chip clasico de Nemesis 2, King's Valley II, Metal Gear 2, etc. 5 canales, los canales 4 y 5 comparten waveform.
051327	Mapper + glue	ASIC de mapeo de MegaROM emparejado con el K051649 en muchos cartuchos.
2212P003	SCC integrado	Version monocapsula: mapper 051327 + generador 051649 en un unico encapsulado.
K052539	SCC-I / SCC+	Version mejorada con 64 KB de DRAM interna y 5 waveforms independientes. Fabricado por Toshiba para Konami.

Por que existio

A mediados de los 80 el sonido estandar del MSX era el AY-3-8910 (PSG): tres canales de onda cuadrada, un generador de ruido y un envolvente de volumen global. Era suficiente para efectos y melodias simples, pero insuficiente para las conversiones de arcade que Konami pretendia. La compania, que ya dominaba el mercado de los MegaROM de 64-128 KB, decidio anadir un chip propio a sus cartuchos de gama alta, compartiendo coste con el mapper de banco. El resultado: cinco voces polifonicas con timbres ricos y la posibilidad de reproducir samples.

Juegos que usaron el SCC original

Lista de titulos con cartucho K051649 (fuente: MSX Wiki, Konami 051649). La columna *RC* es el codigo de producto Konami:

RC	Juego	Ano	Notas
751	Nemesis 2 (Gradius 2)	1987	Cartucho estandar del SCC. Musica: Miki Higashino.
752	F-1 Spirit	1987	Carreras.
755	Salamander (MSX)	1987	Shoot 'em up. Musica: Miki Higashino.
757	Parodius (TwinBee)	1988	Parodia de shoot 'em ups.
758	Gekitotsu Pennant Race	1988	Beisbol.
761	King's Valley 2	1988	Puzzle de plataformas. MSX2.
765	Nemesis 3	1989	MSX2.
768	Pennant Race 2	1989	MSX2.
769	Metal Gear 2: Solid Snake	1990	Infiltracion. MSX2.

Errores frecuentes. Penguin Adventure (1986), King's Valley original, Metal Gear 1 y Vampire Killer **no** usaron SCC: eran PSG puro. Solo sus secuelas o remezclas de fans incorporaron el chip.

El SCC+ (SCC-I)

El K052539 (SCC-I, tambien llamado SCC+ en occidente) se introdujo con *Snatcher* (1988) y se vendio como cartucho independiente (**Sound Cartridge**) acompanando a *Snatcher* y *SD Snatcher*. Es un superconjunto del SCC: en el arranque se comporta como el original, pero un registro de modo permite activar su mapa nativo con **cinco waveforms independientes** de 32 bytes cada una (en vez de cuatro compartidas por ch4/ch5). Esto soluciona el unico limite musical serio del SCC original. *Space Manbow* (1989) es la demostracion canonica de lo que el SCC+ podia hacer.

Hardware moderno

Hoy dia lo habitual para escuchar o programar SCC en hardware real es el **MegaFlashROM SCC+ SD** del equipo RBSC: un cartucho flash que reproduce fielmente el mapper Konami SCC/SCC+, anade ranura SD con Nextor y permite cargar ROMs o reproductores de musica. En emulacion, openMSX es la referencia absoluta: emula el SCC original, el SCC+ y todas las peculiaridades del registro de deformacion documentadas por Manuel Pazos y NYYRIKKI.

CAPITULO 02

Especificaciones tecnicas

Numeros que todo programador SCC debe conocer

Resumen de capacidades

Caracteristica	Valor	Notas
Canales	5	Todos wavetable; en SCC original ch4 y ch5 comparten waveform.
Waveform por canal	32 bytes	Muestras de 8 bits con signo (-128..+127).
Resolucion de frecuencia	12 bits	Rango de periodo 0..4095.
Resolucion de volumen	4 bits	0..15. Logaritmico.
Canal de ruido	No	El ruido se sintetiza cargando una waveform aleatoria.
Generador de envolvente	No	ADSR por software, actualizando el volumen.
Clock maestro	3 579 545 Hz	El mismo clock de CPU/PSG/VDP del MSX.
Registro de activacion	0x9000 = 0x3F	Expone el SCC en 0x9800-0x9FFF (pagina 2).

Clock y formula de frecuencia

El SCC se alimenta del clock principal del MSX: **3 579 545 Hz**. Existen fuentes que citan 1,78 MHz o 1,77 MHz, pero esos son divisores; el valor efectivo para la formula es el de 3,579545 MHz, confirmado por el codigo openMSX (constante 3579545.0 / 32).

El chip recorre sus 32 muestras de waveform una vez por cada (periodo + 1) ticks de clock. De aqui:

$$f_tono = f_clock / (32 \times (periodo + 1))$$

Despejando el registro de periodo para una frecuencia deseada:

$$periodo = (3\ 579\ 545 / (32 \times f_tono)) - 1$$

Ejemplo: La4 (440 Hz) -> periodo = $(3\ 579\ 545 / (32 \times 440)) - 1 = (3\ 579\ 545 / 14\ 080) - 1 \approx 253,4 \rightarrow 253$. Es decir, 0x00FD.

Quirk del periodo. Si periodo ≤ 8 el puntero interno de waveform deja de avanzar (openMSX fuerza `incr = 0`). No se produce tono: se oye el valor de la muestra actual, congelado. *Esa* es la base de toda tecnica de PCM en SCC (ver capitulo 7).

Salida analogica y mezcla

Los cinco canales se suman en el dominio analogico. openMSX, basandose en mediciones con osciloscopio de Jon De Schrijder, modela la salida asi:

$$AmpOut = 640 + Sum((Muestra \times Volumen \text{ AND } 0x7FF0) / 16)$$

El 640 es una offset DC. El rango digital de salida va de +40 a +1235 (11 bits). Desactivar un canal (bit de mixer a 0) equivale a volumen 0. El chip no tiene filtro anti-alias hardware; openMSX anade filtrado digital para acercarse al sonido real.

SCC vs SCC+: la tabla definitiva

	SCC (K051649)	SCC+ (K052539)
Waveform del canal 5	Compartida con canal 4	Independiente, 32 bytes
RAM de waveform total	128 bytes	160 bytes (5 x 32)
RAM interna	Ninguna (solo MegaROM)	64 KB DRAM, ampliable a 128 KB
Modo compatibilidad	n/a	Modo SCC por defecto + modo nativo
Rotacion de ch4 (bit 7 deformacion)	Si	No disponible en SCC+
Forma fisica	Cartucho MegaROM de juego	Sound Cartridge independiente

CAPITULO 03

Mapa de memoria y registros

La hoja de datos que necesitas tener a mano

Donde vive el SCC

El SCC no usa puertos de E/S: sus registros estan **mapeados en memoria**, en la pagina 2 (0x8000-0xBFFF) del slot donde reside el cartucho. Para verlos hay que hacer dos cosas, en este orden:

- Mapear la pagina 2 al slot del cartucho (via BIOS ENASLT en 0x0024).
- Escribir 0x3F en 0x9000: esto pone el mapper Konami en **modo SCC** y expone los registros en 0x9800-0x9FFF.

El mapper Konami SCC (tipo *Konami5*) conmuta bancos de 8 KB en cuatro ventanas:

Ventana	Registro (rango escritura)	Valor al reset
0x4000-0x5FFF	0x5000-0x57FF	banco 0
0x6000-0x7FFF	0x7000-0x77FF	banco 1
0x8000-0x9FFF	0x9000-0x97FF	banco 2 (modo SCC)
0xA000-0xBFFF	0xB000-0xB7FF	banco 3

Gotcha critico (Mideas). Mientras el registro de la ventana P2 (0x9000) contenga un valor con los 6 bits bajos a 0x3F (bancos 0x3F, 0x7F, 0xBF, 0xFF), las lecturas de 0x9800-0x9FFF devuelven registros SCC en vez de ROM. Estos son los *bancos ventana SCC*: un packer de datos nunca debe asignarles bloques de datos porque serian inaccesibles. Se dejan como padding 0xFF (coste: 32 KB de 2 MB).

Mapa de registros del SCC original

Offsets dentro de 0x9800-0x988F. Verificado contra libmsx scc_io.h y BiFi.

Rango	Tam.	Funcion
0x9800-0x981F	32 B	Waveform canal 1
0x9820-0x983F	32 B	Waveform canal 2
0x9840-0x985F	32 B	Waveform canal 3
0x9860-0x987F	32 B	Waveform canal 4 y canal 5 (compartida)
0x9880-0x9881	2 B	Periodo canal 1 (12 bits, low byte primero)
0x9882-0x9883	2 B	Periodo canal 2
0x9884-0x9885	2 B	Periodo canal 3
0x9886-0x9887	2 B	Periodo canal 4
0x9888-0x9889	2 B	Periodo canal 5

Rango	Tam.	Funcion
0x988A	1 B	Volumen canal 1 (low nibble 0..15)
0x988B	1 B	Volumen canal 2
0x988C	1 B	Volumen canal 3
0x988D	1 B	Volumen canal 4
0x988E	1 B	Volumen canal 5
0x988F	1 B	Mixer / on-off: bit0=ch1 .. bit4=ch5 (1 = activo)
0x98A0-0x98BF	32 B	Solo lectura: espejo de la waveform ch5 (= ch4)
0x98E0-0x98FF	32 B	Registro de deformacion (escritura). Leer aqui equivale a escribir 0xFF.

El bloque de periodo/volumen/mixer (0x9880-0x988F) esta **espejado** en 0x9890-0x989F: openMSX aplica address &= 0x0F.

Byte order del registro de periodo

Para escribir un periodo de 12 bits hay que tocar dos registros consecutivos. Confirmado por openMSX setFreqVol:

Registro	Bits usados
Byte bajo (p.ej. 0x9880)	Bits 0..7 del periodo.
Byte alto (p.ej. 0x9881)	Solo el nibble bajo: bits 8..11. El nibble alto se ignora.

Aclaracion de un error comun. Algunas fuentes (p. ej. el wiki japones ngs.no.coocan.jp) afirman que el periodo es de 10 bits. **Es falso.** openMSX aplica `(value & 0xF) << 8 | low_byte`, lo que prueba 12 bits (rango 0..4095). libmsx declara `uint16_t SCC_fdr[5]`.

El registro de deformacion (bits 6 y 7)

El registro en 0x98E0 controla varios modos especiales. Es lo que mucha documentacion llama *registro de-rotacion* o *test register*. Sus bits, medidos por Manuel Pazos en hardware real y recogidos en openMSX:

Bit	Efecto
bit 0	Periodo efectivo de 4 bits: el registro se desplaza >> 8.
bit 1	Periodo efectivo de 8 bits: solo se usa el byte bajo. Si bit0 y bit1 a 1, gana bit1.
bits 2-4	Sin efecto documentado.
bit 5	Reinicia el puntero de waveform a la posicion 0 cada vez que se escribe el registro de periodo.
bit 6	Rotacion automatica de todas las waveforms: la RAM rota una muestra por paso. <i>No se puede escribir</i> en la waveform mientras este activo.

Bit	Efecto
bit 7	Solo SCC original: rota exclusivamente la waveform del canal 4. No funciona en SCC+. Combinarlo con bit6 produce corrupcion de sonido en el chip original.

La velocidad de rotacion la gobierna el periodo del canal 5 (el del canal 4 se ignora en modo rotacion). Escribir `0x00` en `0x98E0` devuelve el chip al modo normal. **No existe** un registro *de-rotate* separado: de-rotar es simplemente escribir 0 en el de deformacion.

Quirks de lectura vs escritura

- **Waveforms 1-3:** al leer `0x9800-0x987F` no se lee la RAM estatica sino el valor de muestra que el chip esta reproduciendo en ese instante (puntero en vivo). Es la base de ciertas tecnicas de PCM.
- **Canal 4/5 compartido:** escribir la waveform del canal 4 la copia al 5 (openMSX: *copy waveform 4 -> waveform 5*). Tienen periodo y volumen independientes, pero **misma forma de onda**.
- **Leer el registro de deformacion** equivale a escribir `0xFF`. Hay que evitar leerlo por accidente.

CAPITULO 04

Deteccion y activacion del chip

Como encontrar el SCC y ponerlo a trabajar

Algoritmo de deteccion fiable

El metodo robusto, parafraseado de libmsx `SCC_inspect()`, se basa en comprobar que la zona `0x9800-0x987F` pasa de ser ROM (solo lectura) a ser RAM (lectura/escritura) cuando se activa el modo SCC:

- Verificar que el slot en `0x8000` es ROM (no RAM).
- Con el SCC **no** expuesto (escribir `0x00` en `0x9000`): la zona `0x9800` debe seguir siendo ROM de solo lectura.
- **Exponer el SCC** (escribir `0x3F` en `0x9000`): ahora `0x9800` debe ser RAM de lectura/escritura. Si sigue siendo de solo lectura, no hay SCC.
- Para distinguir SCC+ de SCC: probar el modo SCC+ (escribir `0x20` en `0xBFFE`, luego `0x80` en `0xB000`) y comprobar si `0xB800` es ahora escribible. Si si -> SCC+; si no -> SCC original.
- Restaurar el mapper escribiendo `0x00` en `0x9000` (y `0xBFFE` si se probó el modo SCC+).

Codigo Z80: activar y silenciar

Esta rutina es la que usa el driver Mideas `test/scc/scc_driver.inc`. Supone que la pagina 2 ya esta mapeada al slot del cartucho:

```

SCC_ENABLE_ADDR equ #9000    ; escribir #3F aqui expone el SCC
SCC_ENABLE_VAL  equ #3F
SCC_WAVE_CH1    equ #9800
SCC_PERIOD_BASE equ #9880    ; 2 bytes por canal, low byte primero
SCC_VOLUME_BASE equ #988A    ; 1 byte por canal, nibble bajo 0..15
SCC_MIXER       equ #988F    ; bits 0..4 activan canales 1..5
SCC_MIXER_MASK  equ #1F

; -----
; SCC_Init: activar el SCC y silenciar los 5 canales.
; Destruye: AF, HL. Preserva: BC, DE, IX, IY.
; -----
SCC_Init:
    ld a, SCC_ENABLE_VAL
    ld (SCC_ENABLE_ADDR), a    ; exponer registros SCC
    xor a
    ld (SCC_MIXER), a          ; mixer = 0 (todos off)
    ld hl, SCC_VOLUME_BASE
    ld (hl), a                 ; vol ch1
    inc hl
    ld (hl), a                 ; vol ch2
    inc hl
    ld (hl), a                 ; vol ch3
    inc hl
    ld (hl), a                 ; vol ch4
    inc hl
    ld (hl), a                 ; vol ch5
    ret

```

Codigo Z80: mapear pagina 2 al cartucho

La pieza que falta. Antes de tocar el SCC hay que garantizar que la pagina 2 del CPU apunta al slot del propio cartucho (suele bastar con copiar el slot de pagina 1, donde se ejecuta el codigo). Usa BIOS RSLREG (0x0138) y ENASLT (0x0024):

```

ENASLT equ #0024 ; A=slot id, H=pagina (bits 6-7), requiere DI
RSLREG equ #0138 ; leer registro de slot primario -&gt; A
EXPTBL equ #FCC1 ; flags de slot expandido (por slot primario)
                ; EXPTBL+4 = SLTTBL (ultimos valores secundarios)

; -----
; enable_page2_cart: paginar la pagina 2 (#8000-#BFFF) al slot de pagina 1.
; Requiere interrupciones desactivadas. Destruye AF, BC, DE, HL.
; -----
enable_page2_cart:
    call    RSLREG
    rrca
    rrca
    and     #03                ; slot primario de pagina 1
    ld      c, a
    ld      b, 0
    ld      hl, EXPTBL
    add     hl, bc
    ld      a, (hl)
    and     #80                ; flag de expandido
    or      c
    ld      c, a
    inc     hl
    inc     hl
    inc     hl
    inc     hl                ; HL = SLTTBL + slot primario
    ld      a, (hl)
    and     #0C                ; bits secundarios de pagina 1
    or      c                ; A = F000SSPP slot id completo
    ld      h, #80            ; pagina 2 (#8000-#BFFF)
    call    ENASLT
    ret

```

Regla de oro. La activacion del SCC (0x3F en 0x9000) **no** es persistente entre accesos: si tu bucle principal o tu driver cambian el banco de pagina 2 por cualquier motivo, debes volver a exponer el SCC antes del siguiente acceso. El backend Mideas salva el banco de P2 en pila y lo restaura despues de cada tick (ver capitulo 10).

CAPITULO 05

Waveforms: el corazon del SCC

Tablas de 32 bytes con signo que definen el timbre

Que es una waveform SCC

Cada canal del SCC reproduce ciclicamente una tabla de **32 muestras de 8 bits con signo** (rango -128 a +127). La forma de esa tabla es el timbre. Una waveform de todo ceros es silencio. Una de +127 durante 16 muestras y -128 durante las otras 16 es una onda cuadrada perfecta. Una secuencia lineal ascendente es una diente de sierra. Y una secuencia de valores pseudoaleatorios es ruido.

Confusion frecuente: **las muestras son de 8 bits, no de 4**. El limite de 4 bits (0..15) se aplica solo al *registro de volumen*, que escala la amplitud de la waveform. La propia waveform tiene la resolution completa de 8 bits.

Rutina Z80 para cargar una waveform

El driver Mideas (SCC_LoadWaveform32) usa LDIR para copiar 32 bytes a la direccion $0x9800 + canal * 32$. En SCC original el canal 5 (indice 4) se redirige al canal 4 porque comparten RAM:

```
; -----
; SCC_LoadWaveform32: copiar 32 bytes de waveform a un canal.
; Entrada: A = indice de canal 0..4. HL = direccion fuente de 32 bytes.
; Destruye: AF, BC, DE, HL. Preserva: IX, IY.
; Nota: solo llamar al cambiar de instrumento, NUNCA por frame.
; -----
SCC_LoadWaveform32:
    cp 4
    jp c, SCC_LoadWaveform32_ch
    ld a, 3          ; SCC original: ch5 (idx 4) comparte #9860 con ch4
SCC_LoadWaveform32_ch:
    rlca
    rlca
    rlca
    rlca
    rlca            ; A = canal * 32 (max 96)
    ld e, a
    ld d, #98        ; DE = #9800 + canal*32
    ld bc, 32
    ldir
    ret
```

Formas de onda estandar

Valores hexadecimales listos para volcar. Recordar: rango -128 (0x80) a +127 (0x7F); el cero es 0x00.

Onda cuadrada (50% duty)

```
; Cuadrada 50%: 16 muestras a +127, luego 16 a -128
square50:
    db #7F,#7F,#7F,#7F,#7F,#7F,#7F,#7F,#7F,#7F,#7F,#7F,#7F,#7F,#7F,#7F
    db #80,#80,#80,#80,#80,#80,#80,#80,#80,#80,#80,#80,#80,#80,#80,#80
```

Triangulo

```
; Triangulo simetrico: -128 .. +127 .. -128
triangle:
    db #80,#91,#A3,#B4,#C5,#D7,#E8,#F9,#0B,#1C,#2D,#3F,#50,#61,#73,#7F
    db #7F,#73,#61,#50,#3F,#2D,#1C,#0B,#F9,#E8,#D7,#C5,#B4,#A3,#91,#80
```

Diente de sierra

```
; Sierra ascendente lineal -128 .. +127 en 32 pasos
saw:
    db #80,#88,#90,#98,#A0,#A8,#B0,#B8,#C0,#C8,#D0,#D8,#E0,#E8,#F0,#F8
    db #00,#08,#10,#18,#20,#28,#30,#38,#40,#48,#50,#58,#60,#68,#70,#7F
```

Seno

```
; Seno: round(127 * sin(2*pi*i/32)), i=0..31
sine:
    db #00,#19,#31,#47,#5A,#6A,#75,#7D,#7F,#7D,#75,#6A,#5A,#47,#31,#19
    db #00,#E7,#CF,#B9,#A6,#96,#8B,#83,#80,#83,#8B,#96,#A6,#B9,#CF,#E7
```

La waveform de seno de arriba es exactamente la que extrae el convertidor `test/scc/vgm2scc.js` del track *Above the Horizon* de Nemesis 2 (index 0 de la biblioteca `nemesis2_waveform_library.json`).

Ruido

```
; Ruido pseudoaleatorio estatico (zumbido tonal):
noise_static:
    db #3A,#C2,#15,#88,#51,#FE,#22,#9C,#40,#B7,#6E,#E1,#08,#D5,#73,#AA
    db #4F,#C8,#19,#86,#5B,#F2,#2D,#A4,#41,#B0,#63,#EE,#07,#D9,#74,#A1

; Para ruido REAL hay que reescribir la waveform en cada frame
; con nuevos valores aleatorios (ver capitulo 7).
```

Curva de volumen SCC (4 bits, logaritmica)

El volumen de 4 bits del SCC no es lineal: cada paso dobla aproximadamente la amplitud. Tabla normalizada (peak = 1.0) usada por el sintetizador de preview de Mideas:

Vol	Amplitud	Vol	Amplitud
0	0.0000	8	0.0891
1	0.0079	9	0.1258
2	0.0112	10	0.1778
3	0.0158	11	0.2512
4	0.0224	12	0.3548
5	0.0316	13	0.5012
6	0.0447	14	0.7079
7	0.0631	15	1.0000

En la practica esto significa que un volumen de 8 ya suena a la mitad de potencia respecto al maximo; los pasos altos (13-15) son donde esta casi toda la expresividad. Un tracker SCC serio debe construir sus curvas ADSR en esta escala logaritmica.

CAPITULO 06

Frecuencia y notas musicales

De La4 (440 Hz) a una tabla de 96 notas

Calcular el periodo de una nota

Recordamos la formula del capitulo 2:

$$\text{periodo} = (3\ 579\ 545 / (32 \times f_{\text{nota}})) - 1$$

Para una nota de la escala temperada, la frecuencia es $f_{\text{nota}} = f_{C0} \times 2^{(n/12)}$, donde $f_{C0} = 16,351597\dots$ Hz y n es el numero de semitonos desde Do0. El driver Mideas genera 96 entradas (notas 0..95, equivalente a C0..B7) con este bucle TypeScript:

```
// buildSccNotePeriodTable() en sccSoundGenerator.ts
const clock = 3579545;
const c0Frequency = 16.351597831287414;
const periods = [];
for (let noteIndex = 0; noteIndex < 96; noteIndex++) {
  const frequency = c0Frequency * Math.pow(2, noteIndex / 12);
  const period = Math.round(clock / (32 * frequency)) - 1;
  periods.push(Math.min(4095, Math.max(0, period)));
}
```

Rutina Z80 para escribir el periodo

SCC_SetPeriod del driver Mideas. Toma indice de canal en A y periodo de 12 bits en DE:

```
; -----
; SCC_SetPeriod: fijar el divisor de frecuencia de 12 bits de un canal.
; Entrada: A = canal 0..4. DE = periodo (low 12 bits).
; Salida: #9880 + canal*2 = byte bajo, siguiente = nibble alto.
; Destruye: AF, HL. Preserva: BC, DE, IX, IY.
; -----
SCC_SetPeriod:
    add a, a          ; canal * 2
    ld hl, SCC_PERIOD_BASE
    add a, l          ; #80 + 0..8 nunca acarrea
    ld l, a
    ld (hl), e        ; byte bajo del periodo
    inc hl
    ld a, d
    and #0F           ; solo nibble bajo del byte alto
    ld (hl), a
    ret
```

Tabla de notas SCC (extracto)

Generada por Mideas y validada en openMSX. Los primeros 9 valores son 0x0FFF (silencio sentinela); la tabla real empieza en A0 (nota 9). Formato little-endian:

```
scc_note_period_table:
; notas 0..8 (sub-audio, sentinela de silencio)
dw #0FFF, #0FFF, #0FFF, #0FFF, #0FFF, #0FFF, #0FFF, #0FFF, #0FFF
; A0 (nota 9) en adelante
dw #0FE3, #0EFE, #0E27, #0D5B, #0C9C, #0BE6, #0B3B, #0A9A
dw #0A01, #0972, #08EA, #086A, #07F1, #077F, #0713, #06AD
; ... 80 entradas mas hasta B7
; A4 (nota 57) = #00FD (253). Lo verifica build_scc_test_rom.mjs.
```

Periodos de prueba del probe Mideas (scc_probe.asm) para una secuencia La4-Do5-Mi5-La5:

Nota	Frecuencia	Periodo	Hex
A4 (La4)	440 Hz	253	0x00FD
C5 (Do5)	523 Hz	213	0x00D5
E5 (Mi5)	659 Hz	169	0x00A9
A5 (La5)	880 Hz	126	0x007E

Semantica de notas en un tracker

En el modelo Mideas (seguimos el de TriloTracker), una celda de patron por canal usa estos valores de nota:

Valor	Significado
0x00..0x5F (0..95)	Nota: indice en la tabla de periodos. C0..B7.
0xFE	Note cut : cortar el sonido del canal inmediatamente.
0xFF	Keep : seguir tocando la nota anterior (sin retrigger).

CAPITULO 07

Tecnicas avanzadas: PCM y ruido

Mas alla del wavetable: samples y sintesis dinamica

Reproduccion de samples (PCM)

El SCC no tiene modo PCM nativo, pero hay tres tecnicas documentadas (el estudio clasico es de NYYRIKKI, preservado en los comentarios del SCC.cc de openMSX):

Metodo 1: actualizacion directa de waveform

Reescribir los 32 bytes de la waveform en cada tick de un timer con las siguientes 32 muestras del sample. Con un bucle Z80 muy apretado se alcanzan ~92 kHz teoricos; en la practica, PCM util a ~7-8 kHz con calidad 4 bits. Simple pero costoso de CPU.

Metodo 2: sample-and-hold por registro de periodo

Truco clave (openMSX lo documenta asi): **escribir el registro de periodo resetea el puntero de waveform y emite inmediatamente el valor de la muestra en la posicion 0**. Si rellenamos los 32 bytes con el mismo valor de muestra y luego escribimos el periodo a toda velocidad, el contador interno no avanza y producimos el mismo sample repetidamente a alta velocidad. Es la base de los players PCM de alta calidad para SCC.

Metodo 3: stop-counter (NYYRIKKI)

Escribir un periodo **menor que 9** congela el contador interno del canal (`incr = 0`). Entonces: (1) parar el canal con periodo < 9, (2) reescribir los 32 bytes con el nuevo valor de sample, (3) reanudar el contador para emitirlo. Es el metodo mas limpio para PCM de calidad.

DPCM de 4 bits

Empaquetar dos muestras de 4 bits por byte (formato SDDD: bit de signo + delta de 3 bits, rango -8..+7). Referencia de implementacion: el proyecto `artrag/PCM-SCC-player-on-msx` en GitHub, que corre en la ISR con un encoder MATLAB.

Sintesis de ruido real

Una waveform aleatoria estatica produce un zumbido tonal (se oye la frecuencia fundamental del periodo). Para ruido blanco **real** hay que reescribir los 32 bytes con nuevos valores aleatorios en cada frame (o varias veces por frame). El coste es el de un LDIR de 32 bytes por canal de ruido. Es como el PSG hacia en hardware: ahora lo hacemos en software.

Vibrato por deformacion de waveform

Otra tecnica clasica: en vez de modulacion de pitch (cambiar el periodo cada frame), se **interpola entre dos waveforms** frame a frame y se reuploada la RAM. Esto produce un efecto de *morphing* timbrico. La comunidad MSX tiene un hilo de 5 paginas dedicado en msx.org (*SCC waveform morphing*).

Combinacion con PSG. El MSX tiene PSG estandar (0xA0-0xA2, 3 canales cuadrados + ruido) mas el SCC (5 wavetable). Los motores musicales tipicos (TriloTracker, Realfun3) conducen ambos en paralelo: PSG para percusion / ruido / bajo, SCC para lead y pads. Polifonia total: **8 voces**.

CAPITULO 08

Motor de musica: envolventes, arpeggios y vibrato

Construyendo un tracker SCC frame a frame

Arquitectura de un row player

Un tracker SCC moderno (siguiendo el modelo TriloTracker que implementa Mideas) tiene esta estructura por canal:

- **Estado por canal** en RAM: nota actual, instrumento, ornamento, volumen base, envolvente de volumen, motor de arpeggio, motor de vibrato, sombra del periodo.
- **Bucle principal:** en cada frame (50/60 Hz, via HALT en el main loop, nunca desde H.TIMI), decrementar un countdown de fila; al llegar a 0, leer la siguiente fila del patron y aplicar cambios (nota, instrumento, ornamento, volumen).
- **Update:** para cada canal activo, recalcular envolvente de volumen, offset de arpeggio y offset de vibrato; recomputar el periodo final; escribir en el SCC solo los registros cuyo shadow ha cambiado.

Envolventes de volumen (ADSR por software)

Cada instrumento lleva una pequena tabla de valores de volumen (0..127 en Mideas, escalada a 0..15 al escribir). El motor recorre la tabla un paso por frame, con posibilidad de **bucle** (sustain) o de mantener el ultimo valor (loop = 0xFF = hold).

```
; Pseudocodigo del motor de envolvente (scc_music_update_envelopes):
; si currentStep < envLen:
;     envOut = envTable[currentStep]
;     currentStep++
; sino si envLoop != 0xFF:
;     currentStep = envLoop      ; sostenido con bucle
; sino:
;     envOut = envTable[envLen-1] ; mantener el ultimo
; volOut = min(envOut, volBase) ; atenuar por volumen base del canal
; escribir volOut en SCC_VOLUME_BASE + canal (solo si cambio)
```

Arpeggios por ornamentos

Un **ornamento** es una tabla de offsets de semitono con signo (p. ej. [0, +4, +7] = acorde mayor). El motor de pitch anade el offset actual a la nota base en cada frame, recomputa el periodo y lo escribe. Asi se construyen arpeggios rapidos de acordes sin necesidad de tres canales.

```

; Ornamento ejemplo: acorde mayor La mayor (La, Do#, Mi)
; 0 = raiz, +4 = tercera mayor, +7 = quinta justo
ornament_major:
    db 3                ; longitud (3 pasos)
    db 0                ; posicion de bucle (0 = desde el principio)
    db 0, 4, 7          ; datos: offsets de semitono con signo

```

IDs de ornamento 1..15; el ID 0 es *sin ornamento* (offset constante 0).

Vibrato (LFO triangular)

Mideas implementa un vibrato por instrumento controlado por tres parametros: **profundidad** (0 = off, hasta 5), **velocidad** (incremento de fase por frame) y **retardo** (frames en silencio tras el note-on antes de empezar). El LFO es una tabla de 64 bytes con un seno de amplitud pico ~31, generado por:

```

// buildSccVibratoTable() en sccSoundGenerator.ts
// 64 muestras: round(31 * sin(2*pi*i/64))
for (let i = 0; i < 64; i++) {
    vib[i] = Math.round(31 * Math.sin(2 * Math.PI * i / 64));
}

```

El motor indexa la tabla con (fase >> 2) & 63 y desplaza aritmeticamente a la derecha por (5 - profundidad). Asi profundidad 5 usa toda la amplitud de la tabla (31 semitonos... en la practica se interpreta como cents de tono), y profundidades menores dividen la excursion entre 2 cada vez.

```

; Pseudocodigo del motor de pitch (scc_music_update_pitch):
; notaFinal = notaBase + offsetOrnamento
; periodoBase = note_period_table[notaFinal]
;
; ; vibrato (solo si ya paso el retardo):
; si vibratoCtr >= vibratoDelay:
;     idx = (vibratoPhase >> 2) & 63
;     offset = scc_vib_table[idx]
;     offset = offset >> (5 - vibratoDepth) ; arithmetic shift
;     vibratoPhase = (vibratoPhase + vibratoSpeed) & 0xFF
; sino:
;     offset = 0
;
; periodoFinal = periodoBase + offset
; si periodoFinal != shadowPeriodo[canal]:
;     escribir periodoFinal en el SCC
;     shadowPeriodo[canal] = periodoFinal

```

Regla de paridad PC/ROM. El sintetizador de preview (components/utils/sccSynthesizer.ts) reimplementa exactamente el mismo motor de pitch en Web Audio, para que lo que oyes en el editor sea lo que sonara en el ROM. Sin esta paridad, ajustar un vibrato a ojo en el editor es imposible.

CAPITULO 09

Layout de datos y API publica

Como se empaqueta una cancion SCC en un ROM

Cabecera de cancion

El backend Mideas parsea cada cancion con esta cabecera binaria (offsets en bytes desde el inicio del bloque de cancion):

Offset	Tipo	Campo
+0	DB	Frames por fila (tempo base)
+1	DB	Longitud de orden (numero de patrones en la lista de orden)
+2	DB	Posicion de restart (para el bucle)
+3	DB	Numero de patrones distintos
+4	DW	Puntero a la tabla de orden
+6	DW	Puntero a la tabla de patrones (entradas: DW puntero + DB numRows)
+8	DW	Puntero a la tabla de instrumentos (32 entradas, IDs 0..31)
+10	DW	Puntero a la tabla de ornamentos (16 entradas, IDs 0..15)

Registro de instrumento (9 bytes)

Offset	Campo
+0	Indice de waveform (en la tabla de waveforms deduplicada)
+1	Volumen por defecto (0..15)
+2	DW puntero a la curva de envolvente
+4	Longitud del envolvente
+5	Punto de bucle del envolvente (0xFF = hold)
+6	Profundidad de vibrato (0..5)
+7	Velocidad de vibrato (fase/frame)
+8	Retardo de vibrato (frames tras note-on)

API publica music_*

El backend SCC expone una API agnostica de chip que el resto del motor de juego llama sin saber si la musica sale por PSG o SCC:

Rutina	Entrada	Funcion
music_init_system	-	Exponer el SCC, inicializar shadows y silenciar todo.
music_play_track	A = track, B bit0 = loop	Cargar una cancion y empezar a reproducirla.
music_stop	-	Silenciar y marcar inactiva.
music_mute	-	Silenciar pero mantener el estado (para pausa).
music_resume	-	Restaurar el sonido tras music_mute.
music_update	-	Avanzar un frame del motor. Llamar desde el main loop, nunca desde H.TIMI.
music_execute_command	varios	Comandos en caliente (cambio de tempo, jump, etc.).

Huella de RAM del runtime

El runtime SCC de Mideas reserva **~130 bytes** de RAM: 17 bytes globales (estado de cancion, punteros, contadores) y 21 bytes por cada uno de los 5 canales (nota, instrumento, envolvente, motor de arpeggio, motor de vibrato, sombra de periodo y volumen). Esta RAM se asigna en el mapa general del proyecto en `variablesGenerator.ts` cuando se detectan tracks SCC.

Restriccion importante. No se puede mezclar musica SCC con musica PSG/PT3 en el mismo ROM. El backend elige uno u otro (`soundGenerator.ts` lo rechaza). Los *efectos de sonido* PSG si siguen disponibles via la API `sfx_*` independiente.

CAPITULO 10

Estudio de caso: el backend SCC de Mideas

Una implementacion completa en produccion

Visión general

El proyecto Mideas incluye un backend SCC completo y nativo, desarrollado en tres fases documentadas. Es un excelente caso de estudio porque cubre toda la cadena: driver Z80 a mano, generador de codigo TypeScript, editor web con preview fiel, convertidor VGM y ROMs de prueba con smoke tests de openMSX.

Las tres fases

Fase	Contenido	Estado
Fase 1	Mapper Konami SCC 2MB como default; driver base (scc_driver.inc) con enable, init, set_period, set_volume, set_mixer, load_waveform32. Probe tecnico que verifica la waveform por readback.	Completa
Fase 2	Generador TypeScript conectado al pipeline; runtime musical por filas; API music_* preservada; manejo del banco P2 (salvar/restaurar 0x3F en cada tick).	Completa
Fase 3	Efectos de tracker de calidad comunidad (modelo TriloTracker): filas de 4 bytes/canal (nota, instrumento, ornamento, volumen); arpeggios por ornamentos; vibrato por instrumento con LFO triangular; motor de pitch con shadow-write por canal; editor con controles de vibrato; preview PC en paridad con el ROM.	Completa (2026-07-17)

Fase	Contenido	Estado
Fase 4	Tecnicas del capitulo 7: instrumentos de ruido real (reescritura de la waveform con bytes pseudoaleatorios en cada frame, tabla ROM de 287 bytes) y morphing de waveforms estilo TriloTracker (motor global, 16 pasos con deltas precalculadas, paso final exacto). Registro de instrumento de 12 bytes (+9 flags, +10 destino, +11 velocidad). Editor y preview en paridad.	Completa (2026-07-18)

Estructura de ficheros

Fichero	Funcion
test/scc/scc_driver.inc	Primitivas del driver Z80 comentadas.
test/scc/scc_probe.asm	Probe tecnico Fase 1: activa el SCC, carga un triangulo en ch1, lo verifica por readback y toca 4 notas.
test/scc/scc_vgm_play.asm	Reproductor de streams VGM (token format compacto).
test/scc/scc_track_test.asm	ROM de test Fase 1+3 con cancion fixture.
test/scc/scc_integrated_check.asm	Runtime + shims music_* (forma MegaROM integrada).
test/scc/vgm2scc.js	Convertidor VGM/VGZ -> stream SCC compacto.
utils/msxGenerator/generators/sccSoundGenerator.ts	Generador TypeScript autoritativo (1763 lineas).
components/utils/sccSynthesizer.ts	Preview Web Audio (505 lineas), paridad de pitch con el ROM.
components/tracker/WaveformEditorModal.tsx	Editor de waveform / instrumento SCC.
utils/audio/sccConstants.js	Constantes canonicas (direcciones, mascara mixer, etc.).
docs/msx/MAPPER_KONAMI_SCC_2MB.md	Documento de diseno del mapper y de las 3 fases.

Token format del reproductor VGM

El convertidor `vgm2scc.js` transforma un VGM con chip K051649 en un stream de tokens compacto para el Z80, de forma que **el Z80 no hace ninguna matematica musical**: solo replay de diffs de estado por frame.

Token	Operandos	Accion
0x00	-	Fin de stream -> bucle al inicio.
0x01	n	Avanzar n frames (writes del frame hechos).
0x02	ch lo hi	SCC_SetPeriod: ch=0..4, periodo 12 bits.
0x03	ch vol	SCC_SetVolume: ch=0..4, vol 0..15.
0x04	mask	SCC_SetMixer.
0x05	ch idx	SCC_LoadWaveform32 desde la entrada idx de la tabla.
0x07	off val	Escritura cruda de byte a 0x9800+off (ajuste parcial).

Verificacion en openMSX

Cada ROM de test tiene un script TCL que arranca openMSX en headless, samplea marcadores RAM en 0xC000-0xC0C1 y valida aserciones:

```
# Ejemplo de smoke test (scc_track_smoke.tcl, esquema):
openmsx / machine C-BIOS_MSX2+ / -romtype KonamiSCC
set rtc_time 0
after time 8 {
    ; C-BIOS tarda ~5s en arrancar el cart
    set active [debug read memory 0xC040] ; scc_music_active
    set mixer [debug read memory 0xC056] ; mixer_shadow
    if {$active == 1 && $mixer != 0} {
        puts "PASS: musica SCC activa y mixer no nulo"
    }
}
after time 20 { exit }
```

Gotchas de openMSX. El open de TCL usa el CWD de openMSX (raiz del repo), no el del shell. Y C-BIOS arranca con 4-5 segundos de retardo emulado: hay que muestrear a `t >= 8`, no antes, o se ve un reset-loop fantasma que no es mas que la secuencia de arranque del BIOS.

CAPITULO 11

Toolchain y ecosistema

Trackers, players, librerias y formatos

Editores / trackers

Herramienta	Autor	Descripcion
TriloTracker (TTSCC)	Trilobyte (cornelisser)	Tracker nativo MSX para PSG+SCC. Variante TTSCC. Formato .TMU. La opcion mas recomendada hoy para componer musica SCC.

Herramienta	Autor	Descripcion
Furnace	tildearrow	Tracker moderno multiplataforma open source. Soporta SCC y SCC+ nativamente como sistema.
Vortex Tracker II	Bulba	Editor Windows para AY/PSG (.PT3). No es SCC nativo, pero se usa para los tracks PSG que acompanan al SCC.
MoonBlaster 1.4	Moonsoft (Remco Schrijvers)	Tracker MSX2 clasico, orientado a MSX-MUSIC y MoonSound. Existe variante con SCC via Sound Cartridge. Formato .MBM.
Realfun3	comunidad	Tracker centrado en SCC-I. Manual PDF disponible.
MSX MuSIC Editor	comunidad	Editor MML multi-chip que corre en MSX real o emulador.

Reproductores y librerias Z80

Reproductor	Autor	Descripcion
TriloTracker Re-player	cornelisser	Reproductor Z80 standalone para PSG+SCC (variante TTSCC). Incluye el compilador TMU que convierte .TMU en un include ASM. Targets Sjasn y asMSX.
VGMPlay-MSX	Grauw (Laurens Holst)	Reproduce VGM en MSX real: rutea los writes 0xD2 al SCC fisico. El reproductor VGM canonico para MSX.
MSXgl (Trilo SCC player)	aoineko	Wrapper C sobre el replayer de Trilo dentro de la MSX Game Library. API: Trilo_Init, Trilo_Play, Trilo_Stop, Trilo_UpdateFrame.
libmsx (SCC device)	mori0091	Libreria C doxygen-documentada. #include <scc.h>: deteccion, activar, frecuencia, volumen, waveform. La API C mas limpia para el chip pelado.
PCM-SCC-player-on-msx	artrag	Reproductor PCM en ISR con encoder MATLAB. Referencia para sample playback.
RoboPlay	ToriHino	Reproductor multiformato MSX; soporta varios formatos de modulo, incluidos los basados en MoonSound.

Compiladores y ensambladores

Herramienta	Autor	Uso
SDCC	comunidad	Compilador C cruzado dominante para Z80. Target -mz80. MSXgl y muchos replayers lo asumen.
Glass	Grauw	Ensamblador Z80 cruzado en Java 8. Sintaxis Z80 estandar, macros regulares y de repeticion, scopes. Usado por VGMPlay-MSX y homebrew moderno.
asMSX	comunidad	Ensamblador cruzado especifico MSX, con todos los opcodes no oficiales.
Sjasn	comunidad	El ensamblador objetivo de TriloTracker y su replayer.
z88dk	comunidad	Toolchain C alternativa (sccz80 + backend SDCC).

Formatos de fichero

Extension	Origen	Contenido
.TMU	TriloTracker	Cancion nativa de TriloTracker (PSG+SCC).
.SCM / .SCC	comunidad	Datos standalone de replay SCC. Sin spec formal unica; el layout lo define el replayer que lo acompaña.
.MBM	MoonBlaster	Cancion de MoonBlaster, orientada a FM (MSX-MUSIC/MoonSound). Atencion: no es SCC nativo salvo en la variante especifica con Sound Cartridge.
.PT3	Vortex Tracker II	Modulo AY/PSG (no SCC). Acompaña al SCC en setups duales.
.VGM / .VGZ	vgmrips	Formato estandar de captura. Comando SCC: 0xD2 aa dd escribe byte dd en registro aa. Offset de cabecera 0x94 = clock SCC.

Aclaracion sobre MBM. MoonBlaster/.MBM **no** es un formato SCC nativo: es FM (MSX-MUSIC o MoonSound). Solo la variante especifica *MoonBlaster for MoonSound + Sound Cartridge* conduce SCC. Para canciones SCC estandar usa .TMU (TriloTracker) o .SCM.

CAPITULO 12

Emulacion y depuracion en openMSX

Como inspeccionar el SCC en tiempo real

openMSX: la referencia

openMSX es el emulador de referencia para trabajo SCC. Emula el SCC original, el SCC+ y **todos** los modos y peculiaridades del registro de deformacion, incluyendo los bugs del bit 7 en el chip original. Su implementacion esta en `src/sound/SCC.cc` y los comentarios incluyen mediciones de osciloscopio de Manuel Pazos, Jon De Schrijder y NYRIKKI.

Inspeccionar el SCC

La consola TCL de openMSX permite watchpoints, dumps de memoria y scripts. Tecnicas utiles para debug SCC:

- **Watchpoint en 0x9800-0x988F:** traza cada escritura de waveform / periodo / volumen con su valor.
- **Dump por frame de la waveform:** un proc TCL que vuelque los 32 bytes de un canal a un fichero cada frame, para ver el morphing o el PCM en accion.
- **Markers RAM:** el codigo bajo test escribe marcadores en posiciones fijas de RAM (p. ej. 0xC000-0xC0C1 en Mideas) para que el smoke lea el estado sin tocar el SCC directamente.
- **Grabacion a WAV:** el sound logger de openMSX permite capturar la salida de audio para comparacion A/B.

Comando de arranque tipico

```
# ROM Konami SCC en openMSX con smoke script TCL:
openmsx -machine C-BIOS_MSX2+ \
  -carta out.rom \
  -romtype KonamiSCC \
  -script smoke.tcl
```

C-BIOS tarda unos 4-5 segundos emulados en arrancar el cartucho: muestrea los marcadores a $t \geq 8$, no antes, para evitar falsos reset-loops.

Otros emuladores

Emulador	Notas
blueMSX	Cycle-accurate, soporta SCC/SCC+. Debugger integrado (desde 2.2.0) con inspeccion de V/RAM. Para SD Snatcher hay que suministrar un SCC.ROM bajo config de machine.
fMSX	Soporta mappers Konami4/Konami5 y SCC.
openMSX (headless + TCL)	El unico viable para smoke tests automatizados en CI.

MoonSound no es SCC. El MoonSound es el chip Yamaha OPL4 (YMF278B), totalmente distinto al SCC. No confundir: el SCC es wavetable de 5 canales con tablas de 32 bytes; el OPL4 es FM + wavetable ADPCM de alta calidad.

APENDICE

Apéndice A: Fuentes y referencias

Bibliografia verificada

Documentacion tecnica canonica

- + MSX Wiki - Konami 051649 (SCC): https://www.msx.org/wiki/Konami_051649
- + MSX Wiki - Konami 052539 (SCC-I/SCC+): https://www.msx.org/wiki/Konami_052539
- + MSX Wiki - Konami Sound Cartridge: https://www.msx.org/wiki/Konami_Sound_Cartridge
- + MSX Wiki - MegaROM Mappers: https://www.msx.org/wiki/MegaROM_Mappers
- + BiFi/msxnet - SCC: <http://bifi.msxnet.org/msxnet/tech/scc.html>
- + BiFi/msxnet - Sound Cartridge (SCC+): <http://bifi.msxnet.org/msxnet/tech/soundcartridge>
- + BiFi/msxnet - MegaROMs: <http://bifi.msxnet.org/msxnet/tech/megaroms>
- + Ultimate MSX FAQ - SCC: <https://www.faq.msxnet.org/scc.html>

Codigo fuente de referencia (emulador y librerias)

- + openMSX SCC.cc (GitHub): <https://github.com/openMSX/openMSX/blob/master/src/sound/SCC.cc>
- + libmsx scc.h: <https://github.com/mori0091/libmsx/blob/master/include/scc.h>
- + libmsx scc.c (deteccion): <https://github.com/mori0091/libmsx/blob/master/src/scc.c>
- + libmsx scc_io.h (direcciones exactas): https://github.com/mori0091/libmsx/blob/master/src/scc_io.h
- + libmsx SCC device docs: https://mori0091.github.io/libmsx/group__SCC_DEVICE.html

Trackers, players y librerias

- + TriloTracker: <https://github.com/cornelisser/TriloTracker>
- + TriloTracker Re-player: <https://github.com/cornelisser/TriloTracker-Re-player>
- + VGMPlay-MSX (Grauw): <https://github.com/grauw/vgmplay-msx>
- + Glass assembler: <https://github.com/grauw/glass>

- + MSXgl: <https://github.com/aoineko-fr/MSXgl>
- + Furnace tracker: <https://tildearrow.org/Furnace/>
- + artrag PCM-SCC-player-on-msx: <https://github.com/artrag/PCM-SCC-player-on-msx>
- + RBSC Konami SCC Cartridge Repair (hardware): <https://github.com/RBSC/Konami-SCC-Cartridge-Repair>

Especificaciones de formato

- + VGM Specification (vgmrips): https://vgmrips.net/wiki/VGM_Specification
- + Furnace SCC/SCC+ doc: <https://tildearrow.org/Furnace/doc/v0.6/7-systems/scc.html>
- + MegaFlashROM SCC+ SD User's Manual (PDF): ver msxcartridgeshop.com

Comunidad y musica

- + MSX Resource Center (foros): <https://www.msx.org/forum/>
- + MSX Assembly Page (Grauw/Sean Young): <https://map.grauw.nl/>
- + SCC waveform morphing thread:
<https://www.msx.org/forum/development/msx-development/scc-waveform-morphing>
- + Big collection free single-cycle waveforms for SCC (foro):
<https://www.msx.org/forum/msx-talk/software/big-collection-free-single-cycle-waveforms-useful-scc>
- + vgmrips Nemesis 2 pack: <https://vgmrips.net/packs/pack/nemesis-2-msx>
- + Miki Higashino (compositora) perfil: <https://www.msxgoto40.com/en/higashino/>